# 1. Periféricos y buses externos

Los procesadores tienen, por regla general, un gran número de pines disponibles para su conexión con dispositivos externos (de hecho, cuentan con varios miles en los modelos más avanzados). Sin embargo, esta disponibilidad tiene sus limitaciones: en primer lugar, no todos los pines son capaces de realizar todas las funciones. En segundo lugar, el uso de pines concretos requiere un grado de planificación del sistema completo a nivel de placa base.

Mucha veces querremos la posibilidad de conectar diferentes dispositivos a nuestro diseño, pero no necesariamente que éstos existan desde el comienzo. Por ejemplo, un microcontrolador sencillo puede utilizarse para controlar diferentes tipos de sensores, actuadores, pantallas, controles y, a nivel de diseño, sólo podremos ofrecer unos pocos pines al usuario.

Para tener la posibilidad de conectar multitud de dispositivos a nuestro procesador, existe el concepto de bus: protocolos que utilizan un conjunto reducido de pines, y permiten, a través de unas pocas líneas de datos, la interconexión de varios (en ocasiones hasta cientos) dispositivos. Dos de los protocolos más extendidos para este propósito son el **I2C** y **SPI**.

## 1.1. I2C

El **Bus I2C** (*Inter-Integrated Circuit* es una de estas alternativas. Es de las más sencillas, y permite, a través de únicamente dos líneas, la conexión de hasta 1000 dispositivos. Los elementos más destacables del bus I2C pueden verse en la Figura 1 y son:

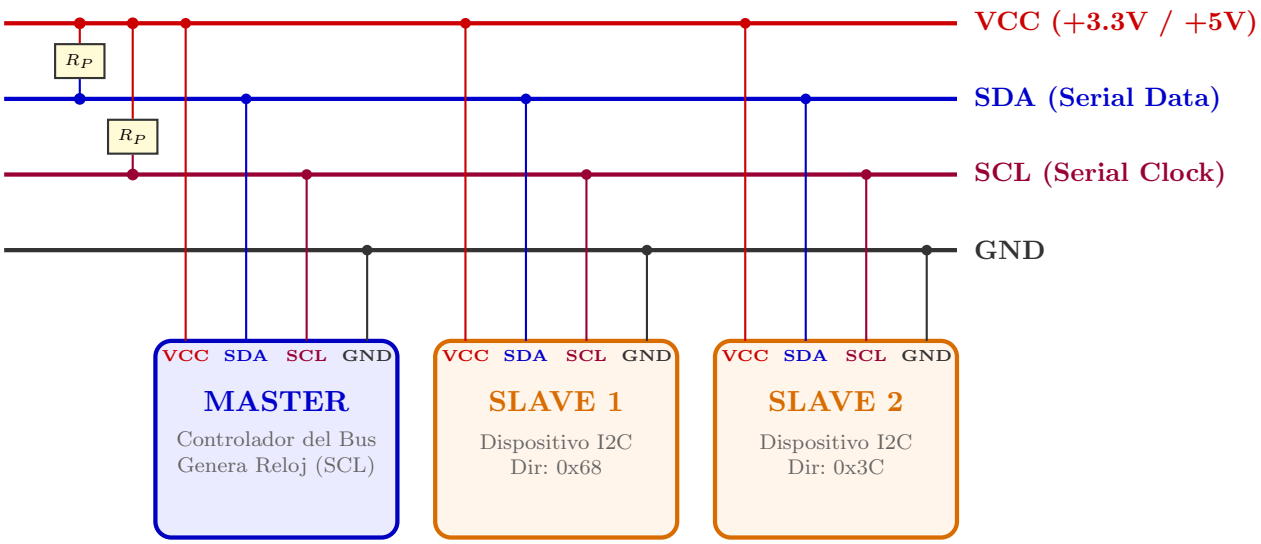



Figura 1: Diagrama de un bus I2C

- *Master*: Un único dispositivo que tiene el control del bus, y genera las peticiones en el mismo. El *Master* puede cambiar con ciertas señalizaciones especiales, pero siempre debe ser un único dispositivo.
- *Slave*: Pueden ser múltiples, son dispositivos que están conectados al bus y escuchan el mismo a la espera de peticiones del *Master*.
- **SDA** o *Serial Data*: Línea de datos serie, a través de la cual se envía la información (bien del *Master* al *Slave* o viceversa).

- **SCL** o *Serial Clock*: Línea de reloj serie. Controlada por el *Master*, lleva la señal de reloj con la que se sincronizan los datos de **SDA**. Generalmente, los dispositivos **I2C** permiten un amplio rango de frecuencias para su funcionamiento, aunque las más típicas son 100 o 400 KHz.

Un dato interesante a observar es el hecho de que tanto **SCL** como **SDA** tienen resistencia de *pull-up* a VCC. Es decir, que por defecto están siempre a "1". Cualquier dispositivo que se conecte, operará en modo de *colector abierto* (open drain). Es decir, que únicamente podrá bajar la línea a "0" conectándola a tierra, enviándose los "unos" pasivamente.

### 1.1.1. Funcionamiento del bus I2C - capa física

Una vez realizadas las conexiones entre los dispositivos, la siguiente pregunta razonable es cómo opera el bus I2C. Las transferencias de datos siguen, de manera general, los siguientes pasos:

1. Una transferencia se inicia con una condición de inicio (**S** (start)), que se señala desde el *Master* bajando **SDA** a "0", mientras **SCL** se mantiene a "1".
2. A continuación **SCL** comienza a oscilar, bajando a "0" en primer lugar, mientras se prepara el bit a enviar en **SDA**.
3. Cuando **SCL** sube a 1, el bit enviado está disponible en **SDA** y el resto de dispositivos *Slaves* pueden leerlo.
4. Se repiten los pasos 2 y 3 tantas veces sea necesario para enviar todos los bits.
5. El último bit es seguido de un pulso de reloj sin dato, en el cual **SDA** se pone a "0".
6. Finalmente, se indica la condición de fin (**P** (stop)), liberando primero **SCL**, y posteriormente **SDA**.

Este proceso se puede ver gráficamente en la Figura 2.

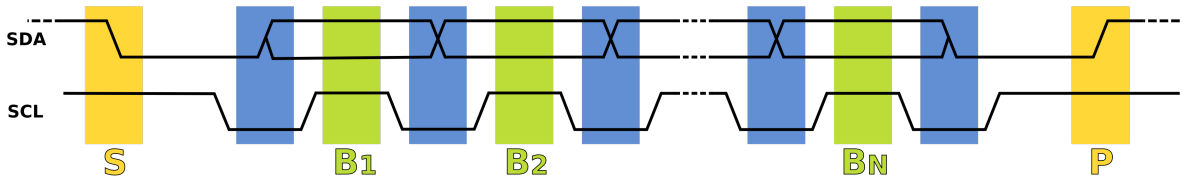



Figura 2: Transferencia de datos en I2C. (fuente:wikipedia)

### 1.1.2. Direccionamiento en I2C - capa lógica

Ahora bien, dentro de estas transferencias, ¿cómo sabemos a qué dispositivo van los bits? para ello, cada *Slave* cuenta con una dirección (de 7 bits, o 10 en versiones más modernas), habitualmente fijada por hardware, y con la que se puede identificar un dispositivo concreto. Así, para escribir, se seguirá el siguiente procedimiento:

1. Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit "0" (write). *Slave* responde **ACK**.
2. Envío del comando (o registro destino) (8 bits). *Slave* responde **ACK**.
3. Envío del dato (8 bits). *Slave* responde **ACK**.
4. Generalmente, y según dispositivo, se pueden enviar datos consecutivos que irán a parar a registros consecutivos si se quiere.
5. Fin de comunicación con condición de parada **P**.

Para leer, el proceso es algo más complejo, ya que primero indicaremos el registro en modo escritura, y posteriormente cederemos el control al *Slave* en modo lectura para que nos pase el dato por el bus.

1. Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit "0" (write). *Slave* responde **ACK**.
2. Envío del comando (o registro destino) (8 bits). *Slave* responde **ACK**.
3. Re-Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit "1" (read). *Slave* responde **ACK**.
4. El *Slave* toma el control y envía el dato de vuelta (8 bits). El *Master* puede responder con **ACK** para pedir más datos consecutivos, o con **NACK** para terminar la comunicación.
5. Fin de comunicación con condición de parada **P**.

Las condiciones **ACK** y **NACK** se indican, respectivamente, con un "0" y un "1" (sí, están invertidas) por parte del receptor, en la línea **SDA** tras las transmisión de un byte.

El proceso para ambas transacciones puede observarse en la Figura 3

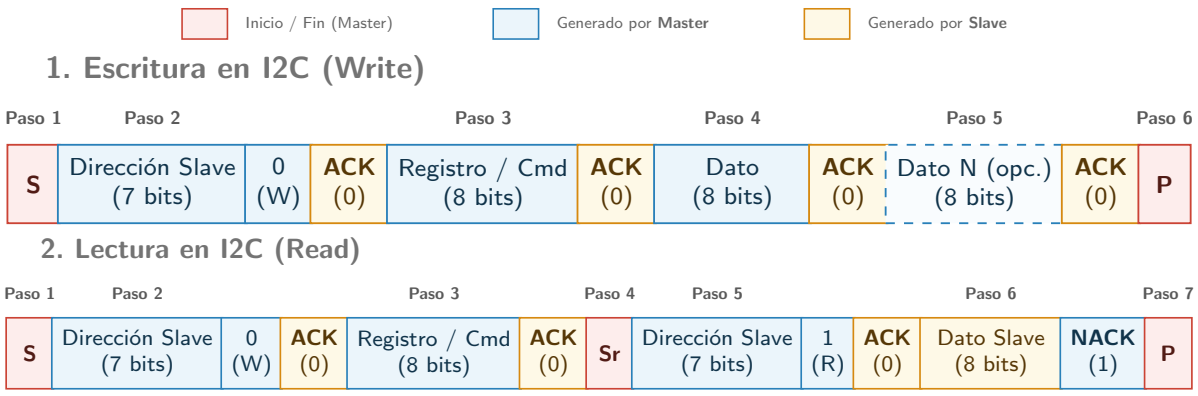



Figura 3: Escrituras y lecturas, respectivamente, en I2C

### 1.1.3. Líneas auxiliares a I2C

En ocasiones, utilizar únicamente las líneas **SCL** y **SDA** no es suficiente. Pensemos por ejemplo en el caso de las interrupciones. Con tan sólo las dos líneas de I2C, capturar una interrupción es imposible, tendríamos que hacer una encuesta activa a los periféricos, preguntando periódicamente si tienen algún dato de interés. Esto sería claramente ineficiente.

Es por ello que las interrupciones de los dispositivos sí van por líneas separadas, para que el procesador sea capaz de capturarlas, identificar directamente el dispositivo causante, y entonces, y sólo entonces, encuestarlo vía I2C para recibir el dato. Adicionalmente, algunos dispositivos disponen de otras señales extra, para por ejemplo habilitarlos o resetearlos.

## 1.2. SPI

El **Bus SPI** (*Serial Peripheral Interface*) es un protocolo de comunicación serie síncrono diseñado para la interconexión de componentes a corta distancia dentro de una misma placa (si bien también se utiliza entre placas cercanas). A diferencia de I2C, SPI es un bus *Full Duplex* (permite transmitir y recibir datos de manera simultánea) y no utiliza un direccionamiento por software, sino líneas físicas dedicadas para seleccionar cada periférico.

Los elementos principales del bus SPI y sus conexiones típicas se resumen en la Figura 4:

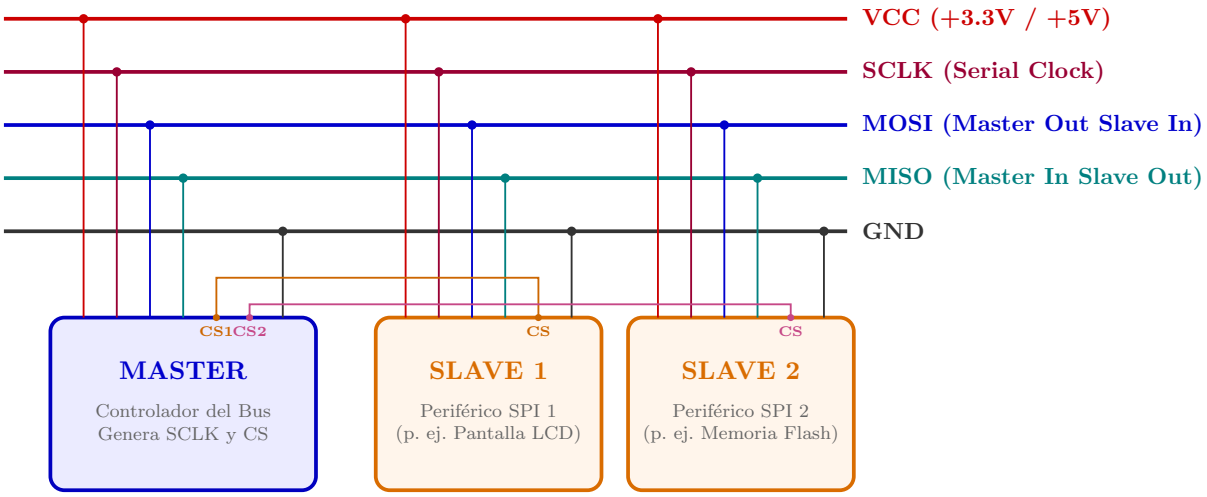



Figura 4: Diagrama de un bus SPI en configuración de selección independiente

- *Master*: Dispositivo controlador que inicia las transacciones, genera la señal de reloj principal (**SCLK**) y gestiona las líneas de selección de cada esclavo **CSx**.
- *Slave*: Dispositivos receptores/emisores. Únicamente están atentos a la información del bus cuando su señal individual de **CSx** está activa. Así se asegura exclusividad en las líneas del bus siempre y cuando el *Master* lo gestione de manera adecuada.
- **SCLK** (*Serial Clock*): Línea de reloj generada por el *Master*. A diferencia de **I2C**, **SPI** no define un límite estricto de velocidad y suele trabajar a frecuencias más altas (en el entorno de MHz).
- **MOSI** (*Master Out Slave In*): Línea serie para el envío de datos desde el *Master* hacia el *Slave*.

- **MISO** (*Master In Slave Out*): Línea serie para la recepción de datos en el *Master* provenientes del *Slave*.
- **CS** (*Chip Select*): Líneas dedicadas e independientes (activas a nivel bajo, "0") para habilitar el *Slave* concreto con el que se desea comunicar.

### 1.2.1. Funcionamiento del bus SPI - capa física

A diferencia de las líneas con *pull-up* de I2C, SPI utiliza salidas activas controladas por el *Master*. Esto elimina la necesidad de las resistencias, permitiendo velocidades mayores (debido a tiempos de conmutación más rápidos).

La información que atraviesa las líneas **MOSI** y **MISO** se almacena con registros de desplazamiento, en los que la unidad de transferencia básica es un byte completo.

Por cada pulso introducido en **SCLK**:

1. El *Master* desplaza un bit hacia el *Slave* por la línea **MOSI**.
2. Simultáneamente, el *Slave* desplaza un bit hacia el *Master* por la línea **MISO**.
3. Tras 8 ciclos de reloj, ambos dispositivos han transferido un byte completo.

**Modos de transmisión SPI (CPOL y CPHA):** Los dispositivos definen su modo de operación en función de dos parámetros configurables en el hardware:

- **CPOL** (*Clock Polarity*): Define el estado de reposo de la línea de reloj **SCLK**. Si CPOL = 0, la línea reposa en "0" lógico; si CPOL = 1, reposa en "1".
- **CPHA** (*Clock Phase*): Define en qué flanco del reloj se capturan los datos. Si CPHA = 0, los datos se muestrean en el primer flanco y se muestrean en el segundo; si CPHA = 1, al contrario.

La combinación de ambos parámetros da lugar a los 4 modos SPI estándar:

| Modo SPI | CPOL | CPHA | Reloj en reposo | Flanco de Muestreo |
|---|---|---|---|---|
| **Modo 0** | 0 | 0 | Nivel Bajo (0) | Flanco de subida |
| **Modo 1** | 0 | 1 | Nivel Bajo (0) | Flanco de bajada |
| **Modo 2** | 1 | 0 | Nivel Alto (1) | Flanco de bajada |
| **Modo 3** | 1 | 1 | Nivel Alto (1) | Flanco de subida |

Tabla 1: Modos de funcionamiento en el protocolo SPI

### 1.2.2. Funcionamiento del bus SPI - capa lógica

Una vez se transmite un byte, su significado no tiene una descripción en el estándar (como sí tiene **I2C**), y depende del fabricante del dispositivo el definir su comportamiento. Habitualmente, el primer byte enviado contendrá el comando que ejecutar o la dirección a la que escribir, mientras que los bytes siguientes serán los datos a transmitir. No obstante es importante recalcar que el funcionamiento específico dependerá del periférico conectado.

### 1.2.3. Topologías de conexión

A nivel de diseño de hardware, existen dos métodos principales para interconectar múltiples periféricos SPI a un mismo controlador:

1. **(*Independent Slave / Parallel Configuration*):** Es la topología estándar vista en la Figura 4. Las líneas de reloj y datos son compartidas por todos los periféricos, pero el controlador necesita una línea **CSx** dedicada para cada esclavo.
2. **(*Daisy Chain Configuration*):** En dispositivos diseñados específicamente para ello, la salida **MISO** de un esclavo se conecta a la entrada **MOSI** del siguiente. De este modo, una única línea **CS** común activa todos los esclavos a la vez. La información se pasa de uno a otro secuencialmente como un único gran registro de desplazamiento.

## 1.3. Dispositivos externos mapeados en memoria (FMC)

En los buses serie como **I2C** o **SPI**, los comandos y datos se envían de forma secuencial, a través de un número reducido de hilos. A nivel de software, esto se traduce en drivers que deben, poco a poco, enviar y recibir toda la información que se desee. Esto es notablemente más complicado que el trabajo con los periféricos integrados del procesador, cuyos registros se mapean directamente en memoria.

Para dispositivos externos con gran cantidad de registros (o directamente, memorias), los microcontroladores actuales suelen incorporar periféricos dedicados que permiten mapear, en el propio espacio de memoria del procesador, el mapa de memoria del dispositivo externo. Esto facilita enormemente la interacción. Por ejemplo, uno muy habitual es el `FMC` (*Flexible Memory Controller*).

Lo hacen conectando el bus interno del procesador directamente con un bus paralelo externo (bus de direcciones, bus de datos y líneas de control) mediante un controlador de memoria intermedio que realiza las traducciones de un espacio de direcciones al otro. Así, el acceso al dispositivo se realiza mediante instrucciones estándar de lectura y escritura en C, exactamente como si se tratara de la memoria RAM o Flash internas. Se muestra un esquema en la Figura 5:

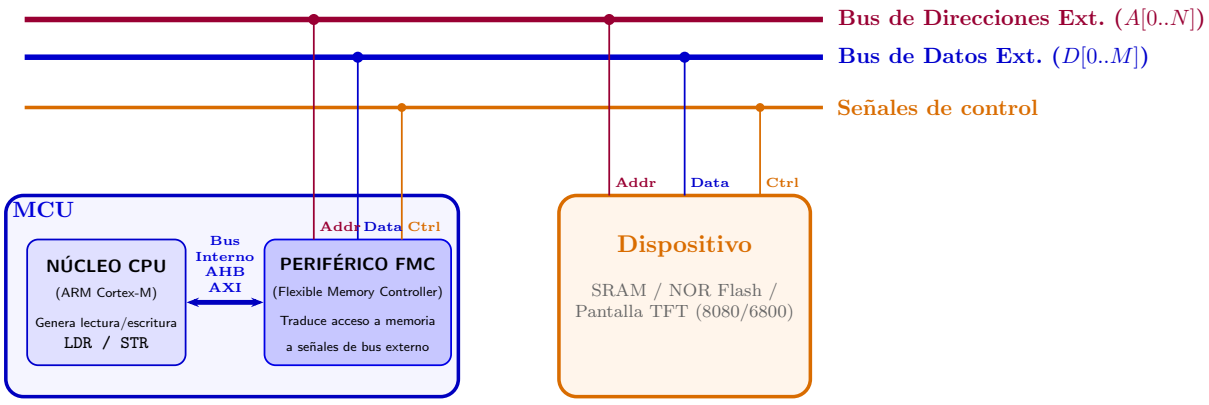



Figura 5: Interconexión paralela mediante controlador FMC con desglose del bus interno de sistema

- **Bus de Direcciones** ($A[0.,25]$)**:** Define la dirección física a la que se desea acceder. El espacio completo puede ocupar decenas de Megabytes.

- **Bus de Datos ($D[0.,15]$ o $D[0.,31]$):** Bus bidireccional de ancho variable. Permite transferir palabras completas en un único ciclo de reloj.
- **Señales de control** Señales variadas, dependientes de la tecnología a la que se conecta (Flash, SRAM, NOR, NAND), que indican la validez de datos/dirección, direccionalidad de la comunicación, activación de periféricos...

### 1.3.1. Acceso por software

La mayor ventaja del mapeo es su facilidad de uso desde software. Para escribir en un controlador de memoria externo basta con realizar asignaciones directas mediante punteros.

Listing 1: Ejemplo de acceso directo a memoria externa mapeada mediante FMC

```
// Direccion base asignada al Banco 1 de la memoria externa
#define EXT_RAM_ADDR  ((volatile uint16_t *) 0x60000000)

void escribir_datos_RAM(void) {
    // Escribe un dato directamente en la SRAM externa mediante el bus FMC
    *(EXT_RAM_ADDR + 0) = 0xABCD; // FMC envía el dato automáticamente
}   //NOTA: por supuesto el FMC debe estar configurado previamente
```

Un caso común de aplicación de controladores FMC es la gestión de pantallas. La pantalla expone una dirección o conjunto de ellas donde enviar la información sobre los píxeles o comandos. Desde el procesador, bastará con mapearlo internamente y escribir ahí, evitando gastar recursos de memoria internos para almacenar las imágenes o gráficos mostrados. Esto es especialmente relevante en microcontroladores con memoria reducida.

Adicionalmente, esto permite que los controladores DMA *Direct Memory Access* transfieran buffers enteros de imágenes directamente a la pantalla a velocidades de decenas de Megabytes por segundo sin intervención de la CPU, mediante ráfagas a los registros mapeados.

## 1.4. Comparativa

Cada una de las interfaces vistas cumple una función diferente dentro del procesador, pero en esencia todas ellas nos permiten la conexión con dispositivos externos. Un resumen de sus características se muestra en la Tabla 2.

| **Periférico** | **I2C** | **SPI** | **FMC** |
|---|---|---|---|
| **Nº de pines** | 2 | 4 | 20-60+ |
| **Conectividad** | Serie | Serie | Paralelo |
| **Velocidad** | 100 kHz – 3.4 MHz | 10 MHz – 100 MHz | 10 MHz – 100 MHz |
| **Ancho de Banda** | kb/s | Mb/s | MB/s |
| **Acceso** | Protocolo estándar (Comandos, ACK) | Protocolo específico (Intercambio de Bytes) | Escritura/Lectura mapeada en memoria |
| **Aplicación principal** | Sensores lentos | Sensores rápidos, memorias y pantallas de baja resolución | Memorias y pantallas de alta velocidad |

Tabla 2: Comparativa general